Conversación con Gemini

Has dicho
queries de grafana para un servidor Windows. Quiero para porcentaje de uso de CPU, uso de RAM y Uso de almacenamiento:# HELP go_build_info Build information about the main Go module.

TYPE go_build_info gauge

go_build_info{checksum="",path="github.com/prometheus-community/windows_exporter",version="v0.31.7+dirty"} 1

HELP go_gc_duration_seconds A summary of the wall-time pause (stop-the-world) duration in garbage collection cycles.

TYPE go_gc_duration_seconds summary

go_gc_duration_seconds{quantile="0"} 0

go_gc_duration_seconds{quantile="0.25"} 0

go_gc_duration_seconds{quantile="0.5"} 0

go_gc_duration_seconds{quantile="0.75"} 0.0005223

go_gc_duration_seconds{quantile="1"} 0.0053868

go_gc_duration_seconds_sum 0.0312196

go_gc_duration_seconds_count 148

HELP go_gc_gogc_percent Heap size target percentage configured by the user, otherwise 100. This value is set by the GOGC environment variable, and the runtime/debug.SetGCPercent function. Sourced from /gc/gogc:percent.

TYPE go_gc_gogc_percent gauge

go_gc_gogc_percent 100

HELP go_gc_gomemlimit_bytes Go runtime memory limit configured by the user, otherwise math.MaxInt64. This value is set by the GOMEMLIMIT environment variable, and the runtime/debug.SetMemoryLimit function. Sourced from /gc/gomemlimit:bytes.

TYPE go_gc_gomemlimit_bytes gauge

go_gc_gomemlimit_bytes 2e+08

HELP go_goroutines Number of goroutines that currently exist.

TYPE go_goroutines gauge

go_goroutines 20

HELP go_info Information about the Go environment.

TYPE go_info gauge

go_info{version="go1.25.10"} 1

HELP go_memstats_alloc_bytes Number of bytes allocated in heap and currently in use. Equals to /memory/classes/heap/objects:bytes.

TYPE go_memstats_alloc_bytes gauge

go_memstats_alloc_bytes 2.863232e+06

HELP go_memstats_alloc_bytes_total Total number of bytes allocated in heap until now, even if released already. Equals to /gc/heap/allocs:bytes.

TYPE go_memstats_alloc_bytes_total counter

go_memstats_alloc_bytes_total 2.948326e+08

HELP go_memstats_buck_hash_sys_bytes Number of bytes used by the profiling bucket hash table. Equals to /memory/classes/profiling/buckets:bytes.

TYPE go_memstats_buck_hash_sys_bytes gauge

go_memstats_buck_hash_sys_bytes 1.474813e+06

HELP go_memstats_frees_total Total number of heap objects frees. Equals to /gc/heap/frees:objects + /gc/heap/tiny/allocs:objects.

TYPE go_memstats_frees_total counter

go_memstats_frees_total 4.450279e+06

HELP go_memstats_gc_sys_bytes Number of bytes used for garbage collection system metadata. Equals to /memory/classes/metadata/other:bytes.

TYPE go_memstats_gc_sys_bytes gauge

go_memstats_gc_sys_bytes 2.869808e+06

HELP go_memstats_heap_alloc_bytes Number of heap bytes allocated and currently in use, same as go_memstats_alloc_bytes. Equals to /memory/classes/heap/objects:bytes.

TYPE go_memstats_heap_alloc_bytes gauge

go_memstats_heap_alloc_bytes 2.863232e+06

HELP go_memstats_heap_idle_bytes Number of heap bytes waiting to be used. Equals to /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.

TYPE go_memstats_heap_idle_bytes gauge

go_memstats_heap_idle_bytes 7.380992e+06

HELP go_memstats_heap_inuse_bytes Number of heap bytes that are in use. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes

TYPE go_memstats_heap_inuse_bytes gauge

go_memstats_heap_inuse_bytes 4.317184e+06

HELP go_memstats_heap_objects Number of currently allocated objects. Equals to /gc/heap/objects:objects.

TYPE go_memstats_heap_objects gauge

go_memstats_heap_objects 34549

HELP go_memstats_heap_released_bytes Number of heap bytes released to OS. Equals to /memory/classes/heap/released:bytes.

TYPE go_memstats_heap_released_bytes gauge

go_memstats_heap_released_bytes 5.644288e+06

HELP go_memstats_heap_sys_bytes Number of heap bytes obtained from system. Equals to /memory/classes/heap/objects:bytes + /memory/classes/heap/unused:bytes + /memory/classes/heap/released:bytes + /memory/classes/heap/free:bytes.

TYPE go_memstats_heap_sys_bytes gauge

go_memstats_heap_sys_bytes 1.1698176e+07

HELP go_memstats_last_gc_time_seconds Number of seconds since 1970 of last garbage collection.

TYPE go_memstats_last_gc_time_seconds gauge

go_memstats_last_gc_time_seconds 1.7812815162774038e+09

HELP go_memstats_mallocs_total Total number of heap objects allocated, both live and gc-ed. Semantically a counter version for go_memstats_heap_objects gauge. Equals to /gc/heap/allocs:objects + /gc/heap/tiny/allocs:objects.

TYPE go_memstats_mallocs_total counter

go_memstats_mallocs_total 4.484828e+06

HELP go_memstats_mcache_inuse_bytes Number of bytes in use by mcache structures. Equals to /memory/classes/metadata/mcache/inuse:bytes.

TYPE go_memstats_mcache_inuse_bytes gauge

go_memstats_mcache_inuse_bytes 2352

HELP go_memstats_mcache_sys_bytes Number of bytes used for mcache structures obtained from system. Equals to /memory/classes/metadata/mcache/inuse:bytes + /memory/classes/metadata/mcache/free:bytes.

TYPE go_memstats_mcache_sys_bytes gauge

go_memstats_mcache_sys_bytes 15288

HELP go_memstats_mspan_inuse_bytes Number of bytes in use by mspan structures. Equals to /memory/classes/metadata/mspan/inuse:bytes.

TYPE go_memstats_mspan_inuse_bytes gauge

go_memstats_mspan_inuse_bytes 93280

HELP go_memstats_mspan_sys_bytes Number of bytes used for mspan structures obtained from system. Equals to /memory/classes/metadata/mspan/inuse:bytes + /memory/classes/metadata/mspan/free:bytes.

TYPE go_memstats_mspan_sys_bytes gauge

go_memstats_mspan_sys_bytes 130560

HELP go_memstats_next_gc_bytes Number of heap bytes when next garbage collection will take place. Equals to /gc/heap/goal:bytes.

TYPE go_memstats_next_gc_bytes gauge

go_memstats_next_gc_bytes 6.054802e+06

HELP go_memstats_other_sys_bytes Number of bytes used for other system allocations. Equals to /memory/classes/other:bytes.

TYPE go_memstats_other_sys_bytes gauge

go_memstats_other_sys_bytes 508947

HELP go_memstats_stack_inuse_bytes Number of bytes obtained from system for stack allocator in non-CGO environments. Equals to /memory/classes/heap/stacks:bytes.

TYPE go_memstats_stack_inuse_bytes gauge

go_memstats_stack_inuse_bytes 884736

HELP go_memstats_stack_sys_bytes Number of bytes obtained from system for stack allocator. Equals to /memory/classes/heap/stacks:bytes + /memory/classes/os-stacks:bytes.

TYPE go_memstats_stack_sys_bytes gauge

go_memstats_stack_sys_bytes 884736

HELP go_memstats_sys_bytes Number of bytes obtained from system. Equals to /memory/classes/total:byte.

TYPE go_memstats_sys_bytes gauge

go_memstats_sys_bytes 1.7582328e+07

HELP go_sched_gomaxprocs_threads The current runtime.GOMAXPROCS setting, or the number of operating system threads that can execute user-level Go code simultaneously. Sourced from /sched/gomaxprocs:threads.

TYPE go_sched_gomaxprocs_threads gauge

go_sched_gomaxprocs_threads 2

HELP go_threads Number of OS threads created.

TYPE go_threads gauge

go_threads 14

HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.

TYPE process_cpu_seconds_total counter

process_cpu_seconds_total 12.875

HELP process_max_fds Maximum number of open file descriptors.

TYPE process_max_fds gauge

process_max_fds 1.6777216e+07

HELP process_open_fds Number of open file descriptors.

TYPE process_open_fds gauge

process_open_fds 382

HELP process_resident_memory_bytes Resident memory size in bytes.

TYPE process_resident_memory_bytes gauge

process_resident_memory_bytes 2.9442048e+07

HELP process_start_time_seconds Start time of the process since unix epoch in seconds.

TYPE process_start_time_seconds gauge

process_start_time_seconds 1.781281075e+09

HELP process_virtual_memory_bytes Virtual memory size in bytes.

TYPE process_virtual_memory_bytes gauge

process_virtual_memory_bytes 2.8688384e+07

HELP promhttp_metric_handler_errors_total Total number of internal errors encountered by the promhttp metric handler.

TYPE promhttp_metric_handler_errors_total counter

promhttp_metric_handler_errors_total{cause="encoding"} 0

promhttp_metric_handler_errors_total{cause="gathering"} 0

HELP promhttp_metric_handler_requests_in_flight Current number of scrapes being served.

TYPE promhttp_metric_handler_requests_in_flight gauge

promhttp_metric_handler_requests_in_flight 1

HELP promhttp_metric_handler_requests_total Total number of scrapes by HTTP status code.

TYPE promhttp_metric_handler_requests_total counter

promhttp_metric_handler_requests_total{code="200"} 89

promhttp_metric_handler_requests_total{code="500"} 0

promhttp_metric_handler_requests_total{code="503"} 0

HELP windows_cpu_clock_interrupts_total Total number of received and serviced clock tick interrupts

TYPE windows_cpu_clock_interrupts_total counter

windows_cpu_clock_interrupts_total{core="0,0"} 55376

windows_cpu_clock_interrupts_total{core="0,1"} 55261

HELP windows_cpu_core_frequency_mhz Core frequency in megahertz

TYPE windows_cpu_core_frequency_mhz gauge

windows_cpu_core_frequency_mhz{core="0,0"} 2294

windows_cpu_core_frequency_mhz{core="0,1"} 2294

HELP windows_cpu_cstate_seconds_total Time spent in low-power idle state

TYPE windows_cpu_cstate_seconds_total counter

windows_cpu_cstate_seconds_total{core="0,0",state="c1"} 543.0986386

windows_cpu_cstate_seconds_total{core="0,0",state="c2"} 0

windows_cpu_cstate_seconds_total{core="0,0",state="c3"} 0

windows_cpu_cstate_seconds_total{core="0,1",state="c1"} 546.9988288

windows_cpu_cstate_seconds_total{core="0,1",state="c2"} 0

windows_cpu_cstate_seconds_total{core="0,1",state="c3"} 0

HELP windows_cpu_dpcs_total Total number of received and serviced deferred procedure calls (DPCs)

TYPE windows_cpu_dpcs_total counter

windows_cpu_dpcs_total{core="0,0"} 38351

windows_cpu_dpcs_total{core="0,1"} 5655

HELP windows_cpu_idle_break_events_total Total number of time processor was woken from idle

TYPE windows_cpu_idle_break_events_total counter

windows_cpu_idle_break_events_total{core="0,0"} 83490

windows_cpu_idle_break_events_total{core="0,1"} 72532

HELP windows_cpu_interrupts_total Total number of received and serviced hardware interrupts

TYPE windows_cpu_interrupts_total counter

windows_cpu_interrupts_total{core="0,0"} 165594

windows_cpu_interrupts_total{core="0,1"} 144012

HELP windows_cpu_logical_processor Total number of logical processors

TYPE windows_cpu_logical_processor gauge

windows_cpu_logical_processor 2

HELP windows_cpu_parking_status Parking Status represents whether a processor is parked or not

TYPE windows_cpu_parking_status gauge

windows_cpu_parking_status{core="0,0"} 0

windows_cpu_parking_status{core="0,1"} 0

HELP windows_cpu_processor_mperf_total Processor MPerf is the number of TSC ticks incremented while executing instructions

TYPE windows_cpu_processor_mperf_total counter

windows_cpu_processor_mperf_total{core="0,0"} 9.668173e+06

windows_cpu_processor_mperf_total{core="0,1"} 8.96709e+06

HELP windows_cpu_processor_performance_total Processor Performance is the average performance of the processor while it is executing instructions, as a percentage of the nominal performance of the processor. On some processors, Processor Performance may exceed 100%

TYPE windows_cpu_processor_performance_total counter

windows_cpu_processor_performance_total{core="0,0"} 2.971272187e+09

windows_cpu_processor_performance_total{core="0,1"} 2.602981206e+09

HELP windows_cpu_processor_privileged_utility_total Processor Privileged Utility represents is the amount of time the core has spent executing instructions inside the kernel

TYPE windows_cpu_processor_privileged_utility_total counter

windows_cpu_processor_privileged_utility_total{core="0,0"} 1.69965543e+08

windows_cpu_processor_privileged_utility_total{core="0,1"} 4.9386093e+07

HELP windows_cpu_processor_rtc_total Processor RTC represents the number of RTC ticks made since the system booted. It should consistently be 64e6, and can be used to properly derive Processor Utility Rate

TYPE windows_cpu_processor_rtc_total counter

windows_cpu_processor_rtc_total{core="0,0"} 2.75683535e+08

windows_cpu_processor_rtc_total{core="0,1"} 2.75683534e+08

HELP windows_cpu_processor_utility_total Processor Utility represents is the amount of time the core spends executing instructions

TYPE windows_cpu_processor_utility_total counter

windows_cpu_processor_utility_total{core="0,0"} 2.97127875e+09

windows_cpu_processor_utility_total{core="0,1"} 2.602987762e+09

HELP windows_cpu_time_total Time that processor spent in different modes (dpc, idle, interrupt, privileged, user)

TYPE windows_cpu_time_total counter

windows_cpu_time_total{core="0,0",mode="dpc"} 2.484375

windows_cpu_time_total{core="0,0",mode="idle"} 545.1875

windows_cpu_time_total{core="0,0",mode="interrupt"} 2.5

windows_cpu_time_total{core="0,0",mode="privileged"} 28.046875

windows_cpu_time_total{core="0,0",mode="user"} 17.375

windows_cpu_time_total{core="0,1",mode="dpc"} 0.171875

windows_cpu_time_total{core="0,1",mode="idle"} 551.625

windows_cpu_time_total{core="0,1",mode="interrupt"} 0.390625

windows_cpu_time_total{core="0,1",mode="privileged"} 20.984375

windows_cpu_time_total{core="0,1",mode="user"} 17.5

HELP windows_exporter_build_info A metric with a constant '1' value labeled by version, revision, branch, goversion from which windows_exporter was built, and the goos and goarch for the build.

TYPE windows_exporter_build_info gauge

windows_exporter_build_info{branch="HEAD",goarch="amd64",goos="windows",goversion="go1.25.10",revision="7fcb3ac32d92d1dcc1c4020ef070e1

HELP windows_exporter_collector_duration_seconds windows_exporter: Duration of a collection.

TYPE windows_exporter_collector_duration_seconds gauge

windows_exporter_collector_duration_seconds{collector="cpu"} 0.0137096

windows_exporter_collector_duration_seconds{collector="logical_disk"} 0.0076119

windows_exporter_collector_duration_seconds{collector="memory"} 0.0076119

windows_exporter_collector_duration_seconds{collector="net"} 0.036412

windows_exporter_collector_duration_seconds{collector="os"} 0.0133686

windows_exporter_collector_duration_seconds{collector="physical_disk"} 0.0137096

windows_exporter_collector_duration_seconds{collector="service"} 0.0744445

windows_exporter_collector_duration_seconds{collector="system"} 0.0076119

HELP windows_exporter_collector_success windows_exporter: Whether the collector was successful.

TYPE windows_exporter_collector_success gauge

windows_exporter_collector_success{collector="cpu"} 1

windows_exporter_collector_success{collector="logical_disk"} 1

windows_exporter_collector_success{collector="memory"} 1

windows_exporter_collector_success{collector="net"} 1

windows_exporter_collector_success{collector="os"} 1

windows_exporter_collector_success{collector="physical_disk"} 1

windows_exporter_collector_success{collector="service"} 1

windows_exporter_collector_success{collector="system"} 1

HELP windows_exporter_collector_timeout windows_exporter: Whether the collector timed out.

TYPE windows_exporter_collector_timeout gauge

windows_exporter_collector_timeout{collector="cpu"} 0

windows_exporter_collector_timeout{collector="logical_disk"} 0

windows_exporter_collector_timeout{collector="memory"} 0

windows_exporter_collector_timeout{collector="net"} 0

windows_exporter_collector_timeout{collector="os"} 0

windows_exporter_collector_timeout{collector="physical_disk"} 0

windows_exporter_collector_timeout{collector="service"} 0

windows_exporter_collector_timeout{collector="system"} 0

HELP windows_exporter_scrape_duration_seconds windows_exporter: Total scrape duration.

TYPE windows_exporter_scrape_duration_seconds gauge

windows_exporter_scrape_duration_seconds 0.0744445

HELP windows_logical_disk_avg_read_requests_queued Average number of read requests that were queued for the selected disk during the sample interval (LogicalDisk.AvgDiskReadQueueLength)

TYPE windows_logical_disk_avg_read_requests_queued gauge

windows_logical_disk_avg_read_requests_queued{volume="C:"} 3.1340589999999997

windows_logical_disk_avg_read_requests_queued{volume="E:"} 0

windows_logical_disk_avg_read_requests_queued{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_avg_write_requests_queued Average number of write requests that were queued for the selected disk during the sample interval (LogicalDisk.AvgDiskWriteQueueLength)

TYPE windows_logical_disk_avg_write_requests_queued gauge

windows_logical_disk_avg_write_requests_queued{volume="C:"} 1.5656367

windows_logical_disk_avg_write_requests_queued{volume="E:"} 0

windows_logical_disk_avg_write_requests_queued{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_free_bytes Free space in bytes, updates every 10-15 min (LogicalDisk.PercentFreeSpace)

TYPE windows_logical_disk_free_bytes gauge

windows_logical_disk_free_bytes{volume="C:"} 4.1420849152e+10

windows_logical_disk_free_bytes{volume="E:"} 4.925161472e+09

windows_logical_disk_free_bytes{volume="HarddiskVolume1"} 5.38968064e+08

HELP windows_logical_disk_idle_seconds_total Seconds that the disk was idle (LogicalDisk.PercentIdleTime)

TYPE windows_logical_disk_idle_seconds_total counter

windows_logical_disk_idle_seconds_total{volume="C:"} 438.1356824

windows_logical_disk_idle_seconds_total{volume="E:"} 441.2862386

windows_logical_disk_idle_seconds_total{volume="HarddiskVolume1"} 441.2862991

HELP windows_logical_disk_info A metric with a constant '1' value labeled with logical disk information

TYPE windows_logical_disk_info gauge

windows_logical_disk_info{disk="0",filesystem="NTFS",serial_number="6E89E38E",type="fixed",volume="HarddiskVolume1",volume_name="Reserva para el sistema"} 1

windows_logical_disk_info{disk="0",filesystem="NTFS",serial_number="C68AF3F1",type="fixed",volume="C:",volume_name=""} 1

windows_logical_disk_info{disk="1",filesystem="NTFS",serial_number="44E7B073",type="fixed",volume="E:",volume_name="Nuevo vol"} 1

HELP windows_logical_disk_read_bytes_total The number of bytes transferred from the disk during read operations (LogicalDisk.DiskReadBytesPerSec)

TYPE windows_logical_disk_read_bytes_total counter

windows_logical_disk_read_bytes_total{volume="C:"} 5.7372672e+07

windows_logical_disk_read_bytes_total{volume="E:"} 0

windows_logical_disk_read_bytes_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_read_latency_seconds_total Shows the average time, in seconds, of a read operation from the disk (LogicalDisk.AvgDiskSecPerRead)

TYPE windows_logical_disk_read_latency_seconds_total counter

windows_logical_disk_read_latency_seconds_total{volume="C:"} 3.1340589999999997

windows_logical_disk_read_latency_seconds_total{volume="E:"} 0

windows_logical_disk_read_latency_seconds_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_read_seconds_total Seconds that the disk was busy servicing read requests (LogicalDisk.PercentDiskReadTime)

TYPE windows_logical_disk_read_seconds_total counter

windows_logical_disk_read_seconds_total{volume="C:"} 3.1340589999999997

windows_logical_disk_read_seconds_total{volume="E:"} 0

windows_logical_disk_read_seconds_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_read_write_latency_seconds_total Shows the time, in seconds, of the average disk transfer (LogicalDisk.AvgDiskSecPerTransfer)

TYPE windows_logical_disk_read_write_latency_seconds_total counter

windows_logical_disk_read_write_latency_seconds_total{volume="C:"} 4.6996956999999995

windows_logical_disk_read_write_latency_seconds_total{volume="E:"} 0

windows_logical_disk_read_write_latency_seconds_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_reads_total The number of read operations on the disk (LogicalDisk.DiskReadsPerSec)

TYPE windows_logical_disk_reads_total counter

windows_logical_disk_reads_total{volume="C:"} 3246

windows_logical_disk_reads_total{volume="E:"} 0

windows_logical_disk_reads_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_requests_queued The number of requests queued to the disk (LogicalDisk.CurrentDiskQueueLength)

TYPE windows_logical_disk_requests_queued gauge

windows_logical_disk_requests_queued{volume="C:"} 0

windows_logical_disk_requests_queued{volume="E:"} 0

windows_logical_disk_requests_queued{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_size_bytes Total space in bytes, updates every 10-15 min (LogicalDisk.PercentFreeSpace_Base)

TYPE windows_logical_disk_size_bytes gauge

windows_logical_disk_size_bytes{volume="C:"} 6.3845695488e+10

windows_logical_disk_size_bytes{volume="E:"} 3.4355544064e+10

windows_logical_disk_size_bytes{volume="HarddiskVolume1"} 5.74619648e+08

HELP windows_logical_disk_split_ios_total The number of I/Os to the disk were split into multiple I/Os (LogicalDisk.SplitIOPerSec)

TYPE windows_logical_disk_split_ios_total counter

windows_logical_disk_split_ios_total{volume="C:"} 281

windows_logical_disk_split_ios_total{volume="E:"} 0

windows_logical_disk_split_ios_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_write_bytes_total The number of bytes transferred to the disk during write operations (LogicalDisk.DiskWriteBytesPerSec)

TYPE windows_logical_disk_write_bytes_total counter

windows_logical_disk_write_bytes_total{volume="C:"} 1.3310976e+07

windows_logical_disk_write_bytes_total{volume="E:"} 0

windows_logical_disk_write_bytes_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_write_latency_seconds_total Shows the average time, in seconds, of a write operation to the disk (LogicalDisk.AvgDiskSecPerWrite)

TYPE windows_logical_disk_write_latency_seconds_total counter

windows_logical_disk_write_latency_seconds_total{volume="C:"} 1.5656367

windows_logical_disk_write_latency_seconds_total{volume="E:"} 0

windows_logical_disk_write_latency_seconds_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_write_seconds_total Seconds that the disk was busy servicing write requests (LogicalDisk.PercentDiskWriteTime)

TYPE windows_logical_disk_write_seconds_total counter

windows_logical_disk_write_seconds_total{volume="C:"} 1.5656367

windows_logical_disk_write_seconds_total{volume="E:"} 0

windows_logical_disk_write_seconds_total{volume="HarddiskVolume1"} 0

HELP windows_logical_disk_writes_total The number of write operations on the disk (LogicalDisk.DiskWritesPerSec)

TYPE windows_logical_disk_writes_total counter

windows_logical_disk_writes_total{volume="C:"} 984

windows_logical_disk_writes_total{volume="E:"} 0

windows_logical_disk_writes_total{volume="HarddiskVolume1"} 0

HELP windows_memory_available_bytes The amount of physical memory immediately available for allocation to a process or for system use. It is equal to the sum of memory assigned to the standby (cached), free and zero page lists (AvailableBytes)

TYPE windows_memory_available_bytes gauge

windows_memory_available_bytes 3.065778176e+09

HELP windows_memory_cache_bytes (CacheBytes)

TYPE windows_memory_cache_bytes gauge

windows_memory_cache_bytes 3.1752192e+07

HELP windows_memory_cache_bytes_peak (CacheBytesPeak)

TYPE windows_memory_cache_bytes_peak gauge

windows_memory_cache_bytes_peak 3.4095104e+07

HELP windows_memory_cache_faults_total Number of faults which occur when a page sought in the file system cache is not found there and must be retrieved from elsewhere in memory (soft fault) or from disk (hard fault) (Cache Faults/sec)

TYPE windows_memory_cache_faults_total counter

windows_memory_cache_faults_total 37110

HELP windows_memory_commit_limit (CommitLimit)

TYPE windows_memory_commit_limit gauge

windows_memory_commit_limit 5.099728896e+09

HELP windows_memory_committed_bytes (CommittedBytes)

TYPE windows_memory_committed_bytes gauge

windows_memory_committed_bytes 1.103867904e+09

HELP windows_memory_demand_zero_faults_total The number of zeroed pages required to satisfy faults. Zeroed pages, pages emptied of previously stored data and filled with zeros, are a security feature of Windows that prevent processes from seeing data stored by earlier processes that used the memory space

(Demand Zero Faults/sec)

TYPE windows_memory_demand_zero_faults_total counter

windows_memory_demand_zero_faults_total 515559

HELP windows_memory_free_and_zero_page_list_bytes The amount of physical memory, in bytes, that is assigned to the free and zero page lists. This memory does not contain cached data. It is immediately available for allocation to a process or for system use (FreeAndZeroPageListBytes)

TYPE windows_memory_free_and_zero_page_list_bytes gauge

windows_memory_free_and_zero_page_list_bytes 2.696093696e+09

HELP windows_memory_free_system_page_table_entries (FreeSystemPageTableEntries)

TYPE windows_memory_free_system_page_table_entries gauge

windows_memory_free_system_page_table_entries 1.2297859e+07

HELP windows_memory_modified_page_list_bytes The amount of physical memory, in bytes, that is assigned to the modified page list. This memory contains cached data and code that is not actively in use by processes, the system and the system cache (ModifiedPageListBytes)

TYPE windows_memory_modified_page_list_bytes gauge

windows_memory_modified_page_list_bytes 9.5629312e+07

HELP windows_memory_page_faults_total Overall rate at which faulted pages are handled by the processor (Page Faults/sec)

TYPE windows_memory_page_faults_total counter

windows_memory_page_faults_total 1.009203e+06

HELP windows_memory_physical_free_bytes The amount of physical memory currently available, in bytes. This is the amount of physical memory that can be immediately reused without having to write its contents to disk first. It is the sum of the size of the standby, free, and zero lists.

TYPE windows_memory_physical_free_bytes gauge

windows_memory_physical_free_bytes 3.065778176e+09

HELP windows_memory_physical_total_bytes The amount of actual physical memory, in bytes.

TYPE windows_memory_physical_total_bytes gauge

windows_memory_physical_total_bytes 4.294422528e+09

HELP windows_memory_pool_nonpaged_allocs_total The number of calls to allocate space in the nonpaged pool. The nonpaged pool is an area of system memory area for objects that cannot be written to disk, and must remain in physical memory as long as they are allocated (PoolNonpagedAllocs)

TYPE windows_memory_pool_nonpaged_allocs_total gauge

windows_memory_pool_nonpaged_allocs_total 0

HELP windows_memory_pool_nonpaged_bytes Number of bytes in the non-paged pool, an area of the system virtual memory that is used for objects that cannot be written to disk, but must remain in physical memory as long as they are allocated (PoolNonpagedBytes)

TYPE windows_memory_pool_nonpaged_bytes gauge

windows_memory_pool_nonpaged_bytes 5.3624832e+07

HELP windows_memory_pool_paged_allocs_total Number of calls to allocate space in the paged pool, regardless of the amount of space allocated in each call (PoolPagedAllocs)

TYPE windows_memory_pool_paged_allocs_total counter

windows_memory_pool_paged_allocs_total 0

HELP windows_memory_pool_paged_bytes (PoolPagedBytes)

TYPE windows_memory_pool_paged_bytes gauge

windows_memory_pool_paged_bytes 6.6838528e+07

HELP windows_memory_pool_paged_resident_bytes The size, in bytes, of the portion of the paged pool that is currently resident and active in physical memory. The paged pool is an area of the system virtual memory that is used for objects that can be written to disk when they are not being used (PoolPagedResidentBytes)

TYPE windows_memory_pool_paged_resident_bytes gauge

windows_memory_pool_paged_resident_bytes 6.0076032e+07

HELP windows_memory_process_memory_limit_bytes The size of the user-mode portion of the virtual address space of the calling process, in bytes. This value depends on the type of process, the type of processor, and the configuration of the operating system.

TYPE windows_memory_process_memory_limit_bytes gauge

windows_memory_process_memory_limit_bytes 1.40737488224256e+14

HELP windows_memory_standby_cache_core_bytes The amount of physical memory, in bytes, that is assigned to the core standby cache page lists. This memory contains cached data and code that is not actively in use by processes, the system and the system cache (StandbyCacheCoreBytes)

TYPE windows_memory_standby_cache_core_bytes gauge

windows_memory_standby_cache_core_bytes 0

HELP windows_memory_standby_cache_normal_priority_bytes The amount of physical memory, in bytes, that is assigned to the normal priority standby cache page lists. This memory contains cached data and code that is not actively in use by processes, the system and the system cache (StandbyCacheNormalPriorityBytes)

TYPE windows_memory_standby_cache_normal_priority_bytes gauge

windows_memory_standby_cache_normal_priority_bytes 9.8115584e+07

HELP windows_memory_standby_cache_reserve_bytes The amount of physical memory, in bytes, that is assigned to the reserve standby cache page lists. This memory contains cached data and code that is not actively in use by processes, the system and the system cache (StandbyCacheReserveBytes)

TYPE windows_memory_standby_cache_reserve_bytes gauge

windows_memory_standby_cache_reserve_bytes 2.71568896e+08

HELP windows_memory_swap_page_operations_total Total number of swap page read and writes (PagesPerSec)

TYPE windows_memory_swap_page_operations_total counter

windows_memory_swap_page_operations_total 151846

HELP windows_memory_swap_page_reads_total Number of disk page reads (a single read operation reading several pages is still only counted once) (PageReadsPerSec)

TYPE windows_memory_swap_page_reads_total counter

windows_memory_swap_page_reads_total 25962

HELP windows_memory_swap_page_writes_total Number of disk page writes (a single write operation writing several pages is still only counted once) (PageWritesPerSec)

TYPE windows_memory_swap_page_writes_total counter

windows_memory_swap_page_writes_total 0

HELP windows_memory_swap_pages_read_total Number of pages read across all page reads (ie counting all pages read even if they are read in a single operation) (PagesInputPerSec)

TYPE windows_memory_swap_pages_read_total counter

windows_memory_swap_pages_read_total 151846

HELP windows_memory_swap_pages_written_total Number of pages written across all page writes (ie counting all pages written even if they are written in a single operation) (PagesOutputPerSec)

TYPE windows_memory_swap_pages_written_total counter

windows_memory_swap_pages_written_total 0

HELP windows_memory_system_cache_resident_bytes The size, in bytes, of the portion of the system file cache which is currently resident and active in physical memory (SystemCacheResidentBytes)

TYPE windows_memory_system_cache_resident_bytes gauge

windows_memory_system_cache_resident_bytes 3.1752192e+07

HELP windows_memory_system_code_resident_bytes The size, in bytes, of the pageable operating system code that is currently resident and active in physical memory (SystemCodeResidentBytes)

TYPE windows_memory_system_code_resident_bytes gauge

windows_memory_system_code_resident_bytes 0

HELP windows_memory_system_code_total_bytes The size, in bytes, of the pageable operating system code currently mapped into the system virtual address space (SystemCodeTotalBytes)

TYPE windows_memory_system_code_total_bytes gauge

windows_memory_system_code_total_bytes 0

HELP windows_memory_system_driver_resident_bytes The size, in bytes, of the pageable physical memory being used by device drivers. It is the working set (physical memory area) of the drivers (SystemDriverResidentBytes)

TYPE windows_memory_system_driver_resident_bytes gauge

windows_memory_system_driver_resident_bytes 9.244672e+06

HELP windows_memory_system_driver_total_bytes The size, in bytes, of the pageable virtual memory currently being used by device drivers. Pageable memory can be written to disk when it is not being used (SystemDriverTotalBytes)

TYPE windows_memory_system_driver_t

Gemini ha dicho

Basándote en las métricas de `windows_exporter` que has compartido, aquí tienes las consultas en **PromQL** para Grafana para obtener los porcentajes de uso de CPU, RAM y Almacenamiento.

1. Porcentaje de Uso de CPU

La métrica `windows_cpu_time_total` es un contador que acumula el tiempo que la CPU pasa en diferentes estados (`idle`, `user`, `privileged`, etc.). Para calcular el porcentaje de uso real, restamos el porcentaje de tiempo que la CPU estuvo inactiva (`mode="idle"`) del 100%.

- **Promedio general por servidor (Instancia):**

  Fragmento de código

  ```
  100 - (avg by (instance) (rate(windows_cpu_time_total{mode="idle"}[5m])) * 100)
  ```

- **Desglosado por cada núcleo (Core) del procesador:**

  Fragmento de código

  ```
  100 - (rate(windows_cpu_time_total{mode="idle"}[5m]) * 100)
  ```

2. Porcentaje de Uso de Memoria RAM

Para la memoria RAM, calculamos la relación entre la memoria total disponible en el sistema (`windows_memory_physical_total_bytes`) y la memoria libre/disponible inmediatamente (`windows_memory_available_bytes`) mediante la fórmula matemática:

Uso de RAM %=TotalTotal−Disponible×100

Consulta para Grafana:

Fragmento de código

```
((windows_memory_physical_total_bytes - windows_memory_available_bytes) /
windows_memory_physical_total_bytes) * 100
```

3. Porcentaje de Uso de Almacenamiento (Disco)

El almacenamiento utiliza las métricas de discos lógicos (`windows_logical_disk_size_bytes` y `windows_logical_disk_free_bytes`). Como en tus métricas aparecen volúmenes del sistema como `HarddiskVolume1`, es recomendable filtrar la consulta utilizando una expresión regular (`volume=~"[A-Z]:"`) para que solo muestre las unidades con letra asignada (como `C:` o `E:`).

La operación matemática es equivalente a:

Uso de Almacenamiento %=Taman˜o TotalTaman˜o Total−Espacio Libre×100

Consulta para Grafana (Filtrado por unidades de disco principales):

Fragmento de código

```
((windows_logical_disk_size_bytes{volume=~"[A-Z]:"} -
windows_logical_disk_free_bytes{volume=~"[A-Z]:"}) /
windows_logical_disk_size_bytes{volume=~"[A-Z]:"}) * 100
```

Si quieres ver absolutamente todos los volúmenes sin discriminar por letra, puedes remover el filtro de la siguiente manera:

Fragmento de código

```
((windows_logical_disk_size_bytes - windows_logical_disk_free_bytes) /
windows_logical_disk_size_bytes) * 100
```

💡 Consejos para la configuración en Grafana:

1. En el panel de visualización lateral derecho de Grafana, asegúrate de configurar la unidad (**Standard options -> Unit**) en **Percent (0-100)**.
2. En las propiedades de la leyenda (**Legend**), puedes usar expresiones como `{{instance}}` o `{{volume}}` para identificar correctamente qué servidor o disco estás visualizando en tus gráficas.

Has dicho
este me muestra dos resultados: ((windows_memory_physical_total_bytes - windows_memory_available_bytes) / windows_memory_physical_total_bytes) * 100

Gemini ha dicho

Es completamente normal que te muestre dos resultados (o incluso más si tuvieras más servidores). Esto sucede porque **Prometheus** está recolectando esa misma métrica desde dos fuentes o "etiquetas" distintas en tu entorno.

Para solucionarlo y ver un único resultado limpio, debes entender por qué pasa y cómo agruparlo.

¿Por qué salen dos resultados?

Si miras la leyenda o los detalles de los dos resultados en Grafana, notarás que se diferencian en sus **etiquetas** (labels). Las dos razones más comunes son:

1. **Tienes dos servidores (o instancias):** Si estás monitoreando más de un servidor Windows, la métrica se duplica: una para `instance="servidor1"` y otra para `instance="servidor2"`.
2. **Duplicidad por `job`:** A veces, un mismo servidor es recolectado por dos tareas distintas en Prometheus (por ejemplo, `job="windows-exporter"` y `job="integrations/windows"`).

Cómo solucionarlo en Grafana

Dependiendo de lo que quieras lograr, aplica una de las siguientes soluciones en tu consulta:

Opción A: Filtrar por un servidor específico (Recomendado para Dashboards)

Si estás armando un tablero donde seleccionas el servidor mediante una variable (usualmente llamada `$instance` o `$server`), añade el filtro entre las llaves `{}`:

Fragmento de código

```
((windows_memory_physical_total_bytes{instance="$instance"} -
windows_memory_available_bytes{instance="$instance"}) /
windows_memory_physical_total_bytes{instance="$instance"}) * 100
```

(Si no usas variables, puedes escribir el nombre directamente: `{instance="tu-servidor:9182"}`).

Opción B: Agrupar por Instancia (Si quieres ver ambos, pero bien identificados)

Si quieres que Grafana te muestre los dos resultados pero asegurándote de que Prometheus los trate de forma independiente y ordenada por servidor, puedes usar un agregador como `avg`:

Fragmento de código

```
avg by (instance) (((windows_memory_physical_total_bytes - windows_memory_available_bytes) /
windows_memory_physical_total_bytes) * 100)
```

🛠️ El toque final en Grafana

Para que no aparezca un texto largo y confuso en la gráfica, ve a las propiedades del panel en la derecha, busca la sección **Standard options -> Display name** (o **Legend** en versiones anteriores) y escribe:

`{{instance}}`

Esto hará que cada línea o indicador muestre únicamente el nombre del servidor correspondiente.

¿Qué etiquetas o nombres diferentes te aparecen en esos dos resultados que estás viendo?

Has dicho
el servidor es 10.2.7.66

Gemini ha dicho

¡Perfecto! Sabiendo que el servidor específico es `10.2.7.66`, podemos acotar las consultas para que **Grafana te muestre un único resultado exacto**, eliminando la duplicidad.

Dependiendo de cómo esté registrado en tu Prometheus (si incluye el puerto o no), la etiqueta `instance` suele tomar el formato `10.2.7.66:9182` (si usa el puerto por defecto de *windows_exporter*) o simplemente `10.2.7.66`.

Para asegurarnos de que capture el servidor en cualquiera de los dos casos, utilizaremos el operador de expresiones regulares `=~` y un comodín `.*`.

Aquí tienes las 3 consultas definitivas y limpias para tu servidor:

1. Uso de RAM (Resultado Único)

Fragmento de código

```
((windows_memory_physical_total_bytes{instance=~"10.2.7.66.*"} -
windows_memory_available_bytes{instance=~"10.2.7.66.*"}) /
windows_memory_physical_total_bytes{instance=~"10.2.7.66.*"}) * 100
```

2. Uso de CPU (Resultado Único)

Esta consulta promedia los núcleos para darte el uso total de la CPU de ese servidor en específico:

Fragmento de código

```
100 - (avg(rate(windows_cpu_time_total{instance=~"10.2.7.66.*", mode="idle"}[5m])) * 100)
```

3. Uso de Almacenamiento (Por cada disco de ese servidor)

Esta consulta te mostrará el uso de cada unidad (C:, E:, etc.) perteneciente **únicamente** a la IP `10.2.7.66`:

Fragmento de código

```
((windows_logical_disk_size_bytes{instance=~"10.2.7.66.*", volume=~"[A-Z]:"} -
windows_logical_disk_free_bytes{instance=~"10.2.7.66.*", volume=~"[A-Z]:"}) /
windows_logical_disk_size_bytes{instance=~"10.2.7.66.*", volume=~"[A-Z]:"}) * 100
```

🛠️ Configuración recomendada en Grafana para estos paneles:

1. **Nombre de la Leyenda (Legend):**
    - Para los paneles de **RAM** y **CPU**, ve a la barra derecha de configuraciones, busca **Standard options -> Display name** (o la sección *Legend*) y escribe: `10.2.7.66`. Así la gráfica tendrá un nombre limpio.
    - Para el panel de **Almacenamiento**, escribe `{{volume}}`. De este modo, si el servidor tiene un disco C: y un disco E:, la gráfica te mostrará una línea etiquetada como `C:` y otra como `E:`.
2. **Unidad de Medida (Unit):**

    En los tres paneles, busca la opción **Unit** en el menú de la derecha y selecciona **Percent (0-100)**.

Gemini ha dicho

¡Excelente captura! Al ver la imagen queda perfectamente claro el problema de la duplicidad.

Si te fijas en los textos pequeños debajo de los porcentajes, el servidor `10.2.7.66:9182` está siendo recolectado bajo dos **jobs** (tareas de Prometheus) diferentes:

1. El gráfico de la izquierda (`36.7%`) viene de **`job="windows-client"`**.
2. El gráfico de la derecha (`55.2%`) viene de **`job="windows-exporter"`**.

Como la consulta matemática que estás usando no discrimina el `job`, Prometheus te devuelve el cálculo para ambos.

Para solucionarlo y quedarte con **un solo gráfico** que muestre el valor real, debes elegir cuál de los dos *jobs* es el correcto (usualmente el oficial es `windows-exporter`, pero depende de cómo lo tengas configurado).

Aquí tienes las consultas corregidas especificando el `job` para que Grafana te muestre un único resultado:

Opción 1: Si el job correcto es `windows-exporter` (Recomendado)

Fragmento de código

```
((windows_memory_physical_total_bytes{instance=~"10.2.7.66.*", job="windows-exporter"} -
windows_memory_available_bytes{instance=~"10.2.7.66.*", job="windows-exporter"}) /
windows_memory_physical_total_bytes{instance=~"10.2.7.66.*", job="windows-exporter"}) * 100
```

Opción 2: Si el job correcto es `windows-client`

Fragmento de código

```
((windows_memory_physical_total_bytes{instance=~"10.2.7.66.*", job="windows-client"} -
windows_memory_available_bytes{instance=~"10.2.7.66.*", job="windows-client"}) /
windows_memory_physical_total_bytes{instance=~"10.2.7.66.*", job="windows-client"}) * 100
```

💡 Consejo extra para limpiar la interfaz de Grafana:

Una vez que reemplaces la query en tu panel, desplázate en el menú de la derecha de Grafana hasta encontrar la sección **Standard options** -> **Display name** (o la propiedad **Legend** abajo de la consulta, según tu versión de Grafana) y escribe simplemente: `RAM Utilizada` o `{{instance}}`

Esto quitará todo ese texto largo de `{instance=..., job=...}` que aparece abajo del número central y dejará el panel completamente limpio y profesional.

Has dicho
display name para el almaceanmiento

Gemini ha dicho

Para el panel de **Almacenamiento (Disco)**, la configuración del **Display name** (o **Legend**) es un poco diferente a la de la RAM, ya que aquí sí te interesa distinguir qué disco estás viendo (por ejemplo, la unidad `C:` de la `E:`).

En el menú de la derecha de Grafana, busca la opción **Standard options -> Display name** (o la sección **Legend** justo debajo de la query si estás en una versión anterior) y utiliza la siguiente variable nativa de Prometheus:

Plaintext

```
Disco {{volume}}
```

¿Por qué usar esto?

Al escribir `Disco {{volume}}`, Grafana extraerá dinámicamente la letra del volumen de la métrica (`volume="C:"`, `volume="E:"`) y renderizará un texto limpio y fácil de leer debajo de cada indicador o gráfica:

- **Disco C:**
- **Disco E:**

⚠️ Importante para la Query de Almacenamiento

Recuerda aplicar también el filtro del `job` que descubrimos en el paso anterior para que no se te dupliquen los discos. Tu consulta definitiva de almacenamiento para este servidor debe quedar así:

Fragmento de código

```
((windows_logical_disk_size_bytes{instance=~"10.2.7.66.*", job="windows-exporter", volume=~"[A-Z]:"} - windows_logical_disk_free_bytes{instance=~"10.2.7.66.*", job="windows-exporter", volume=~"[A-Z]:"}) / windows_logical_disk_size_bytes{instance=~"10.2.7.66.*", job="windows-exporter", volume=~"[A-Z]:"}) * 100
```

(Nota: Cambia `job="windows-exporter"` por `job="windows-client"` si decidiste usar el otro recolector).

Has dicho
ahora necesito el query para la creación de una alerta de uso de cpu. Este es un ejemplo para una maquina Linux

Gemini ha dicho

Para crear la regla de alerta de uso de CPU en tu servidor Windows, el enfoque cambia un poco en comparación con Linux (donde se suele usar `node_cpu_seconds_total` y el modo `idle`). En Windows, como vimos, calculamos el uso restando el tiempo de inactividad (`mode="idle"`) del total.

Para una regla de alerta en **Prometheus** (o para configurarla directamente en **Grafana Alerting**), necesitamos que la condición sea verdadera cuando el uso supere un umbral (por ejemplo, el **90%**) durante un tiempo determinado (por ejemplo, **5 minutos**).

Aquí tienes el ejemplo de la query de alerta adaptado a tu servidor Windows y a los `jobs` de tu entorno:

🚨 Consulta para la Alerta (PromQL)

Fragmento de código

```
100 - (avg by (instance, job) (rate(windows_cpu_time_total{instance=~"10.2.7.66.*", job="windows-exporter", mode="idle"}[5m])) * 100) > 90
```

*(Nota: Si prefieres aplicar esta alerta a **todos** los servidores Windows de tu infraestructura en lugar de solo a la IP `.66`, simplemente remueve el filtro de la instancia dejando `windows_cpu_time_total{job="windows-exporter", mode="idle"}`).*

📄 Ejemplo de Configuración Completa (Formato Prometheus Rules)

Si defines tus alertas mediante código en los archivos de configuración de Prometheus (`alert.rules.yml`), la estructura siguiendo tu ejemplo de Linux sería la siguiente:

YAML

```
groups:
  - name: WindowsServerAlerts
    rules:
      - alert: WindowsCPUHighUsage
        expr: 100 - (avg by (instance, job) (rate(windows_cpu_time_total{job="windows-exporter",
mode="idle"}[5m])) * 100) > 90
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "Uso de CPU elevado en el servidor {{ $labels.instance }}"
          description: "El uso de CPU en la instancia {{ $labels.instance }} ha superado el 90%
durante los últimos 5 minutos. Valor actual: {{ printf \"%.2f\" $value }}%"
```

🛠️ ¿Cómo configurarlo si usas Grafana Alerting?

Si estás creando la alerta desde la interfaz gráfica de Grafana dentro de tu panel de CPU:

1. **Expression / Query (A):** Coloca la query PromQL de arriba (sin el `> 90`).
2. **Define alert condition (B):** Añade una expresión de tipo **Reduce** (para obtener el último valor `Last`) y luego una expresión de tipo **Math** que sea `$B > 90`.
3. **For:** Configura `5m` (así evitas falsos positivos por picos de uso momentáneos).
4. **Notifications:** En los textos de notificación puedes usar `{{ $values.A.Value }}` para incluir el porcentaje exacto de CPU que disparó la alarma en el mensaje de Teams, Slack o Correo.

Has dicho
no, lo quiero para Windows server. Este es el ejemplo de linux: 100 - (avg by (instance)

(rate(node_cpu_seconds_total{instance="10.2.7.5:9100",

mode="idle"}[2m])) * 100)

Gemini ha dicho

¡Entendido perfectamente! Basándonos exactamente en tu ejemplo de Linux, aquí tienes la traducción idéntica adaptada para **Windows Server** utilizando las métricas de tu `windows_exporter` y manteniendo la misma lógica estructural (promedio por instancia y ventana de tiempo de 2 minutos).

Dado que en tu consulta anterior vimos que tienes la IP `10.2.7.66` y que el recolector correcto es `windows-exporter`, la consulta exacta que debes usar es la siguiente:

Consulta de Alerta para Windows Server (PromQL)

Fragmento de código

```
100 - (avg by (instance) (rate(windows_cpu_time_total{instance="10.2.7.66:9182", job="windows-
exporter", mode="idle"}[2m])) * 100)
```

🔍 Comparativa de qué cambió respecto a tu ejemplo de Linux:

Para que veas la equivalencia exacta entre ambos sistemas:

Componente	En Linux (Tu ejemplo)	En Windows (Tu consulta)
Métrica base	`node_cpu_seconds_total`	`windows_cpu_time_total`
Puerto por defecto	`:9100`	`:9182`
Filtro de inactividad	`mode="idle"`	`mode="idle"` *(Es idéntico)*
Filtro adicional	No requiere	`job="windows-exporter"` *(Obligatorio en tu caso para evitar la duplicidad con el otro job)*

🛠️ ¿Cómo aplicarla en la alerta de Grafana?

Si estás usando el sistema de alertas de Grafana, pondrás esta query en el paso 1, y en el paso de la condición matemática (**Math**) añadirás el umbral de disparo, por ejemplo: `$A > 90` (para activarse si supera el 90%).

